

Comdove WhatsApp Mock Server — Hackathon PRD

2026-09-19 · @Someone

1. Purpose

Build a fake WhatsApp server that impersonates the Meta Cloud API, so Comdove (wat-backend) can be tested end to end without a real Meta account, real phone numbers, or API costs. Comdove switches to the mock by changing one environment variable (the Graph API base URL). Nothing else in wat-backend changes.

The mock has two halves: a **server** that speaks Meta's API and fires Meta-shaped webhooks back at Comdove, and a **browser client grid** where testers act as WhatsApp end users, each tile owning its own number.

Why this matters: today every test of sending, receiving, and message-status handling needs a real WhatsApp number and Meta's sandbox limits. With the mock, anyone on the team can register 5 to 10 business numbers, simulate 10 to 15 customers per number, and watch every message flow live.

2. Goals and non-goals (v1)

Goals

- Emulate the Meta Cloud API send endpoint for **text messages only**, with real Meta response bodies and error codes.
- Fire Meta-shaped webhooks to Comdove: inbound customer messages and status updates (sent, delivered, read), signed with X-Hub-Signature-256.
- Register 5 to 10 business numbers on the mock server; customer numbers live in **client groups** of up to 10 numbers each.
- Browser client: pick a group, get a tile per number; human-driven send and receive, per-tile online/offline toggle with offline queuing.
- Group lifecycle: chat history and undelivered messages persist across closing and reopening a group; the pending delivered/read webhooks fire on reconnect; only one browser session can hold a group at a time.
- Basic auto-reply per tile (echo / keyword map) to simulate a talking customer.

- Admin view: register numbers and groups, watch the live message log (from, to, text, status).

Non-goals for v1

- Media, templates, interactive buttons/lists, reactions.
- 24-hour session window enforcement.
- Playwright or any automated test suite (phase 2; v1 only prepares for it).
- Multi-device per number, group chats, encryption.

3. System overview

Three parts. Comdove (wat-backend) is the system under test and is not modified beyond one env variable.

```

flowchart LR
    CB[Comdove wat-backend<br/>system under test] -->|POST| MS[Mock Meta Server<br/>API emulator + registry]
    MS -->|signed webhooks<br/>inbound + statuses| CB
    MS <-->|WebSocket| CG[Client Grid<br/>browser tiles = customers]
    AD[Admin UI<br/>register numbers, message log] --> MS
  
```

Reading: Comdove sends a text message to the mock exactly as it would to graph.facebook.com. The mock delivers it over WebSocket to the tile owning the recipient number (or queues it if that tile is offline) and posts status webhooks back to Comdove. When a tester types a reply in a tile, the mock posts an inbound-message webhook to Comdove, signed the same way Meta signs.

One repo, two apps: a Node/TS/Express server (matches the wat-backend stack) and a React/Vite client (matches wat-frontend). Admin UI can be a page inside the client app pointed at admin endpoints.

4. Components

Five components. Each can be one owner during the hackathon.

#	Component	What it does	Key interfaces
C1	Meta API emulator	Accepts POST <code>/[phone_number_id]/messages</code> (type text), validates token and recipient, returns Meta-shaped success and error JSON	HTTP, same paths as Graph API v-current
C2	Webhook dispatcher	Posts inbound messages and status events (sent, delivered, read) to Comdove's webhook URL in Meta's entry/changes/value envelope, signed with X-Hub-Signature-256, retries on non-200; includes late statuses when a group reconnects	Outbound HTTP to Comdove
C3	Registry, groups + presence	Stores business numbers (phone_number_id, token), client groups (name, up to 10 customer numbers), the per-group session lock, per-tile online flags, and offline queues	SQLite store (survives group disconnects)
C4	Client grid (browser)	Group picker, then the group's tiles. Tile per customer number: chat history, send box, online toggle, auto-reply mode (manual / echo / keyword map)	One WebSocket session per group
C5	Admin UI + control API	Register/delete numbers and groups, live message log (from, to, text, status timeline), reset button; same actions exposed as plain HTTP endpoints for future automation	HTTP + WebSocket for live log

Data model stays small: **numbers** (business and customer), **groups** (name, member numbers, session lock), **conversations** (pair of numbers), **messages** (id, from, to, body, status timeline, timestamps), **presence** (group connected plus per-tile flag).

5. Functional requirements

ID	Requirement	Priority
FR-01	Admin can register 5 to 10 business numbers; each gets a phone_number_id and a fake bearer token	Must
FR-02	POST /{phone_number_id}/messages with type text returns Meta's success shape: messaging_product, contacts[] with wa_id, messages[] with a wamid-style id	Must
FR-03	Emulator returns real Meta error JSON (error.code, error.subcode, fbtrace_id) for: invalid token (401), unknown phone_number_id, unregistered recipient, malformed payload, and one rate-limit code	Must
FR-04	A message to an online tile in a connected group appears in that tile in under 1 second	Must
FR-05	A message to an offline tile or a closed group is queued; coming back online flushes the queue in order	Must
FR-06	Server fires status webhooks to Comdove: sent on accept, delivered on client receipt, read when the chat is opened	Must
FR-07	A tester typing in a tile fires an inbound-message webhook to Comdove in Meta's envelope	Must
FR-08	All webhooks signed with X-Hub-Signature-256 using a configurable app secret; retried up to 3 times on non-200	Must
FR-09	Client launch page lists registered groups with free/locked status; opening a free group renders one tile per number in that group	Must
FR-10	Per-tile auto-reply modes: manual (default), echo, keyword map (small editable table: contains X, reply Y), optional fixed delay	Should

ID	Requirement	Priority
FR-11	Admin live log: every message with from, to, body, and status timeline, updating in real time; late statuses from reconnecting groups appear the same way	Must
FR-12	POST /reset wipes all messages and queues (numbers and groups optionally kept)	Must
FR-13	Control API: register number, create group, set presence, inject an inbound message, all as plain HTTP for future test automation	Should
FR-14	Group state is URL-addressable, e.g. /client?group=alpha, opening straight into that group (subject to the lock)	Should
FR-15	Client groups: customer numbers are organized into named groups of up to 10 numbers, registered on the server	Must
FR-16	One active session per group: a group with a live WebSocket session shows as locked on the launch page and a second browser cannot open it; the lock releases on disconnect	Must
FR-17	Per-group persistence: closing the group's browser and reopening it restores full chat history plus any messages queued while it was closed	Must
FR-18	Statuses for messages queued during a group closure fire on reconnect: delivered on flush, read when the chat is opened, visible in the admin log like live traffic	Must

Must = demo blocks without it. Should = do it if the hackathon clock allows; design for it either way.

6. Message flows

Outbound: Comdove sends to a customer

```
sequenceDiagram
    participant CB as Comdove
    participant MS as Mock Server
    participant T as Client Tile
    CB->>MS: POST /{phone_number_id}/messages (text)
    MS-->>CB: 200 + wamid
    MS->>CB: webhook status=sent
    alt tile online
        MS->>T: deliver over WebSocket
        MS->>CB: webhook status=delivered
        T->>MS: user opens chat
        MS->>CB: webhook status=read
    else tile offline
        MS->>MS: queue message
        T->>MS: tile toggled online
        MS->>T: flush queue in order
        MS->>CB: webhook status=delivered
    end
end
```

Inbound: customer replies

```
sequenceDiagram
    participant T as Client Tile
    participant MS as Mock Server
    participant CB as Comdove
    T->>MS: send text (WebSocket)
    MS->>CB: signed webhook, messages[] envelope
    CB-->>MS: 200 (else retry x3)
    Note over T,CB: auto-reply mode does the same,<br/>triggered by an incoming message
```

Ordering rule: per conversation, messages and their statuses are delivered in send order. Across conversations, no ordering guarantee (same as Meta).

Group presence. A closed group counts as offline for every number in it. On reconnect the server flushes each conversation's queue in order, fires the pending delivered webhooks, and fires read per chat as the tester opens it. The admin log shows these late statuses exactly like live traffic, which is the observability the group concept exists for.

7. What it should look like

Client grid (/client)

- Launch page: list of registered groups with name, number count, and status (free, or locked by an active session). Pick a free group to enter; a locked group is greyed out.
- Inside a group: a responsive grid of tiles, one per number in the group, 3 to 4 per row on a laptop. Header bar: group name, connection state, count of online tiles.
- Each tile: the number as title, a green/grey online dot that is also the toggle, an unread badge, a scrollable chat history (incoming left, outgoing right, WhatsApp-style), a text input with send, and a small gear for auto-reply mode.
- Reopening a group restores every tile's full chat history plus anything queued while it was closed.
- Keep it plain. No avatars, no theming beyond readable chat bubbles. The point is density: a full group of 10 tiles workable on one screen.

Admin (/admin)

- Top: number registration form (display number, label) and the list of registered numbers with their phone_number_id, token, type (business/customer), and claim status.
- Middle: live message log table, newest first: time, from, to, text, status chips (sent / delivered / read), webhook result (200 or retry count).
- Corner: Reset button with a confirm.

Config

Mock server reads Comdove's webhook URL and app secret from env. Comdove points GRAPH_API_BASE_URL at the mock. That is the entire integration surface.

8. Build constraints (non-negotiable)

These exist so phase 2 (Playwright automation, load testing) costs nothing later.

1. **Meta contract fidelity over features.** Copy request/response shapes from Meta's Cloud API docs for the text-message slice. If the shape is wrong, the mock tests nothing.
2. **data-testid on every interactive element** in both client grid and admin, from the first commit.
3. **URL-addressable groups** (FR-14) so a future test can open N browser contexts straight onto specific groups.
4. **Control API before UI.** Every admin action must work as a plain HTTP call first; the admin UI calls the same endpoints. No behavior that only exists behind a button.

5. **Stateless enough to reset.** POST /reset returns the system to a known state in one call.
6. **Stack:** Node/TS/Express server, React/Vite/TS client, WebSocket (ws or socket.io), SQLite (group history and queues must survive disconnects). Matches the Comdove repos so anyone on the team can maintain it.
7. **Scale target:** 10 business numbers x 15 customers = 150 numbers, a few messages per second. No performance engineering needed; do not block it either (no per-message polling).

9. Out of scope for v1 (planned later)

- Media, template, and interactive message types.
- 24-hour customer service window and template-only enforcement.
- Failure injection beyond the FR-03 error set (chaos mode, duplicate webhooks, out-of-order delivery).
- Scripted load generation and the Playwright test suite (phase 2; the control API and testids are their landing pad).
- Auth on the mock's admin/control endpoints. It runs on localhost or an internal network only. Never expose it publicly.

10. Hackathon plan and acceptance

Suggested split (3 people)

Owner	Lane	Deliverable
Dev A	C1 + C2: API emulator, webhook dispatcher, signatures	Comdove can send and receive against the mock
Dev B	C3 + C5: registry, presence, queue, control API, admin log	Numbers registered, traffic visible live
Dev C	C4: client grid, tiles, presence toggle, auto-reply	10+ tiles workable in one browser

Start from a shared contract: agree the WebSocket event names and the control API paths in the first 30 minutes, write them in the repo README, then build in parallel.

Demo script (this is the acceptance test)

- ☐ Register 5 business numbers and two client groups of up to 10 customer numbers each in admin.
- ☐ Point a local wat-backend at the mock via env; no code change in wat-backend.
- ☐ From Comdove, send a text to an online tile: it appears in under 1 second, admin log shows sent -> delivered -> read, Comdove receives all three status webhooks.
- ☐ Toggle a tile offline, send to it, toggle online: queued message arrives, delivered fires.
- ☐ Close a group's browser tab, send it 2 to 3 messages from Comdove, reopen the group: full history plus the queued messages appear, delivered webhooks fire on reconnect, read fires when the chat is opened.
- ☐ While a group is open, try opening it from a second tab: launch page shows it locked and refuses.
- ☐ Type a reply in a tile: Comdove receives a signed inbound webhook and shows the message.
- ☐ Set one tile to keyword-map auto-reply and hold a short bot conversation from Comdove.
- ☐ Trigger one error case (bad token) and show Comdove handling Meta's error JSON.
- ☐ POST /reset, everything clears, numbers and groups remain.

If all boxes tick, v1 is done and phase 2 (Playwright + load) can start on top without rework.